Huấn luyện trên GPU

Kiểm đúng trước, đo tài nguyên sau

Giống kiểm TLE/MLE trong CP

Input nhỏ đúng chưa đủ cho input lớn.

Nhưng input nhỏ sai thì đo GPU chưa giúp ích.

Overfit một batch → debug → ước lượng → đo thật.

Ba bước đầu chạy trên CPU; bước cuối cần GPU được cấp.

01

Một batch phải học được

Bốn góc, nhãn theo dấu

Mạng 2→8→2, không dropout/augmentation.

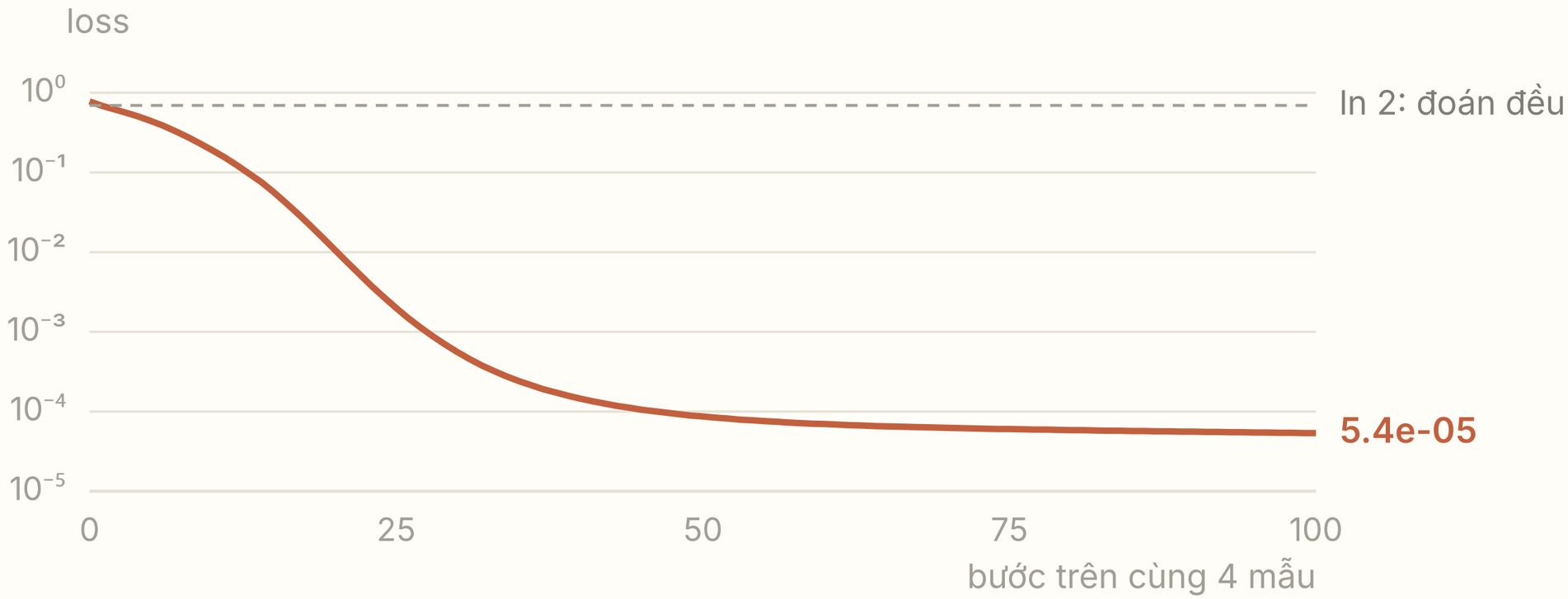
100 bước trên cùng batch.

KẾT QUẢ

```
loss: 0.776305 -> 5.4e-05
one-batch accuracy: 1.0
```

Điểm trên chính batch đã học, không phải validation.

Loss 100 bước



Overfit một batch bốn mẫu (code bài): loss đi từ gần ln 2 xuống cỡ 10^{-5} sau 100 bước, chứng tỏ forward, loss, backward và step đã nối đúng.

Overfit một batch ảnh thật



bước 0: loss 2.4, đúng 1/16



bước 20: loss 0.53, đúng 16/16



bước 50: loss 0.038, đúng 16/16



bước 200: loss 0.0024, đúng 16/16

Overfit một batch 16 ảnh Fashion-MNIST: lặp lại trên cùng batch, loss phải về gần 0 và cả 16 ảnh được đoán đúng.

HỎI NHANH

Sau 100 bước trên bốn mẫu, loss giảm từ 0.776 xuống 5.4e-05, và accuracy bằng 1.0. Ta kết luận được gì?

A Model sẽ đạt accuracy cao trên tập validation

B Có thể bỏ validation, vì loss đã gần bằng 0

C Forward, loss, backward và bước cập nhật đã được nối đúng

D Learning rate 0.05 cũng sẽ hợp với dữ liệu thật

HỎI NHANH

Thử cho model học thuộc 16 ảnh của một bài 10 lớp. Loss đứng yên ở 2.3026 suốt 200 bước, accuracy cũng không đổi. Nên kiểm gì trước?

A Thêm dữ liệu, vì 16 ảnh là quá ít để học

B Optimizer có nắm đúng tham số không, và có gọi `step()` không

C Tăng dropout để model bớt học thuộc batch

D Giảm batch xuống 1 cho gradient rõ hơn

Tìm nơi lỗi xuất hiện đầu

Input → activation → logits → loss → gradient.

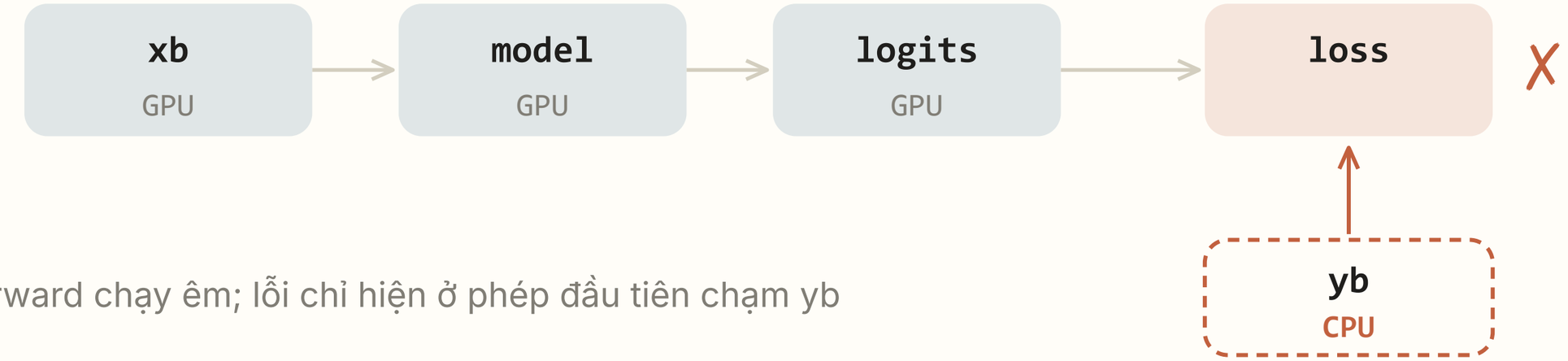
`torch.isfinite` phát hiện NaN và inf.

KẾT QUẢ

```
finite rows: [False, True]
bad row indices: [0]
```

Input đã NaN: đổi optimizer không sửa dữ liệu.

Cùng một thiết bị



forward chạy êm; lỗi chỉ hiện ở phép đầu tiên chạm yb

Model và xb đã ở GPU nên forward chạy được; target yb còn ở CPU nên lỗi device chỉ hiện ra ở dòng tính loss.

HỎI NHANH

Đã gọi `model.to('cuda')` và `xb = xb.to('cuda')`. Forward chạy được, nhưng dòng `loss = criterion(logits, yb)` báo các tensor không nằm trên cùng thiết bị. Vì sao?

A Model mới lên GPU một phần, phải gọi lại `.to`

B `yb` vẫn ở CPU, phải chuyển nó sang cùng thiết bị

C Loss phải tính ở CPU, nên đưa logits về CPU

D `yb` phải đổi sang float cho khớp với logits

HỎI NHANH

Logits có shape `(4, 3)`, target tạo bằng `torch.tensor([0., 2., 1., 1.])`. `nn.CrossEntropyLoss()` báo `expected scalar type Long but found Float`. Sửa thế nào?

A Đổi logits sang `.long()` cho cùng kiểu

B Đổi loss sang `BCEWithLogitsLoss`

C Tạo target kiểu `long`, chẳng hạn bằng `.long()`

D Đưa target về shape `(4, 3)` bằng `.view`

Bốn hợp đồng

Shape có tên chiều · dtype phù hợp loss.

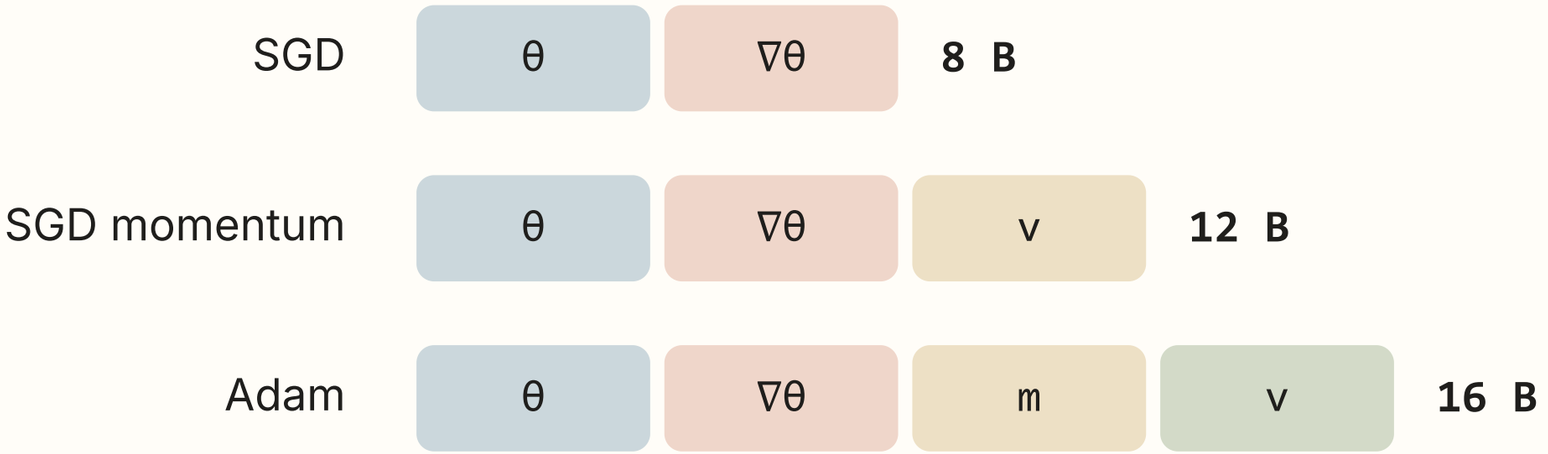
Model/tensor cùng device · gradient còn đường nối.

Anomaly detection chỉ bật khi debug vài bước.

02

Bộ nhớ không chỉ có weight

Byte mỗi tham số



mỗi ô 4 byte (FP32), tính cho một tham số; activation tính riêng

Mỗi tham số FP32 cần 4 byte cho trọng số, 4 cho gradient, cộng trạng thái optimizer: SGD 8 byte, SGD momentum 12, Adam 16.

HỎI NHANH

Đổi Adam sang SGD có momentum, tức mỗi tham số giữ một buffer vận tốc, và vẫn dùng FP32. Phần weight, gradient và trạng thái optimizer còn khoảng bao nhiêu byte cho mỗi tham số?

A 4

B 8

C 12

D 16

Mạng nhỏ và tensor lớn

KẾT QUẢ

```
parameters: 42
weight bytes: 168
```

Weight, gradient, hai moment với Adam: 672 byte.

Score `(16,8,1024,1024)` FP32:

512 MiB nếu lưu đủ cả tensor.

Chỉ tính kích thước, không cấp phát tensor ấy.

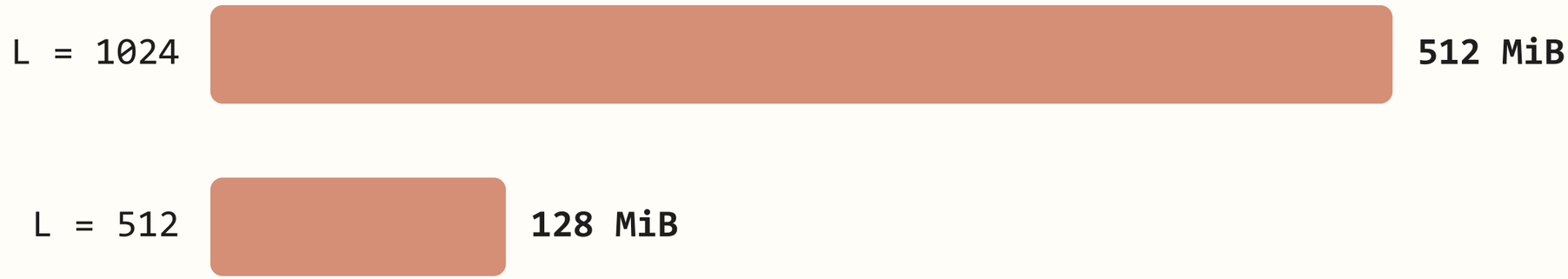
Giảm batch, giảm độ dài

Batch 64 → 32: input và activation giảm khoảng một nửa.

Weight, gradient, moment: giữ nguyên.

Score `(16,8,L,L)`: 1024 → 512 thì 512 MiB → 128 MiB.

Bộ nhớ tỉ lệ với



score (16, 8, L, L) FP32: L giảm một nửa, bộ nhớ giảm bốn lần

Tensor attention score (16, 8, L, L) ở FP32: 512 MiB với L = 1024 nhưng chỉ 128 MiB với L = 512, vì kích thước tỉ lệ với L^2.

HỎI NHANH

Giảm batch size đi một nửa, giữ nguyên kiến trúc và optimizer. Phần bộ nhớ nào giảm theo, trong khi weights vẫn đứng yên?

A Input và activation của batch

B Mọi phần cùng giảm đúng một nửa

C Weights và hai moment của Adam

D Chỉ bộ nhớ đệm của allocator

OOM và graph sống

Giảm batch, độ dài, kích thước ảnh trước.

`empty_cache` không xóa tensor còn sống.

`history.append(loss)` kéo theo graph.

Validation quên `no_grad` mà giữ loss: graph giữ cả activation.

Metric scalar: lưu `loss.item()`.

Gradient accumulation

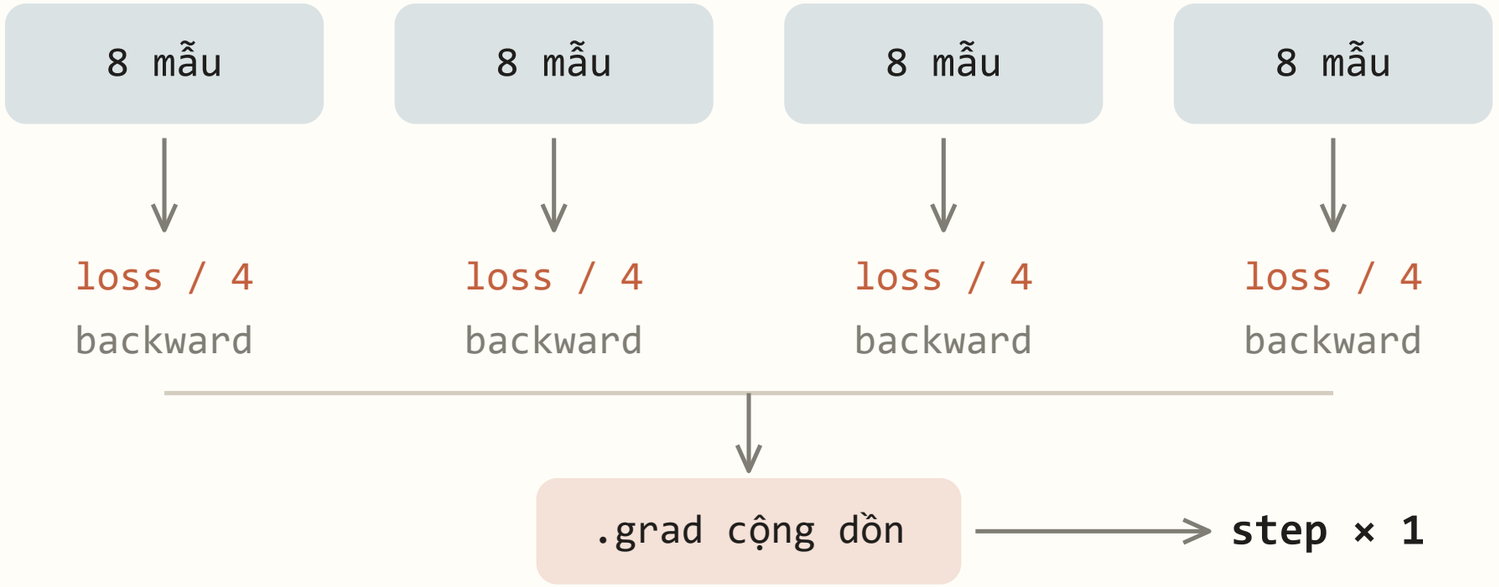
```
for x_part, y_part in zip(X_acc.chunk(4), y_acc.chunk(4)):
    micro_loss = criterion(net(x_part), y_part)
    (micro_loss / 4).backward()   # .grad cộng dồn
```

KẾT QUẢ

```
divide by 4: True | norm ratio: 1.0 | matches full batch: True
divide by 4: False | norm ratio: 4.0 | matches full batch: False
```

Bốn microbatch 8 mẫu so với một batch 32.

Bốn microbatch, một step



chia 4: tỉ lệ 1.0

không chia: 4.0

so với gradient batch 32

Gradient accumulation: bốn microbatch 8 mẫu, mỗi cái backward với loss chia 4, .grad cộng dồn rồi mới step một lần; kết quả trùng gradient của batch 32.

Accumulation: điều kiện và cái giá

Loss mỗi microbatch chia .

Microbatch bằng nhau; BatchNorm vẫn chỉ thấy microbatch.

Activation chỉ cỡ một microbatch.

lượt forward/backward cho một step: tiết kiệm bộ nhớ, không tăng tốc.

HỎI NHANH

Batch 64 bị OOM, batch 16 thì chạy được. Ta dùng accumulation 4 microbatch, mỗi microbatch 16 mẫu, trên 6400 mẫu train. Mỗi epoch có bao nhiêu lần `backward()` và bao nhiêu lần `optimizer.step()`?

A 100 lần backward, 100 lần step

B 400 lần backward, 400 lần step

C 100 lần backward, 400 lần step

D 400 lần backward, 100 lần step

03

Mixed precision và CUDA

Mixed precision và GradScaler

Autocast: matmul/conv FP16; softmax, log, tổng FP32.

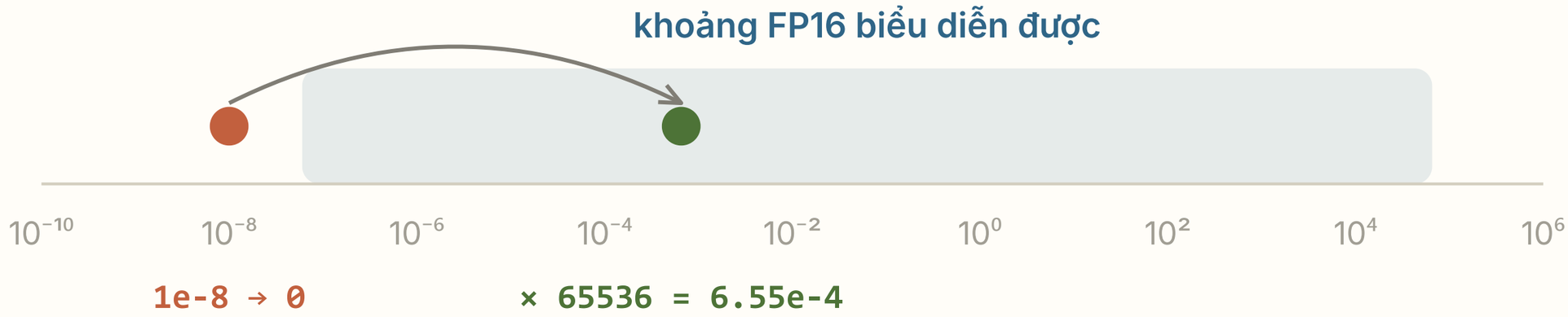
GradScaler nhân loss với 65536 trước backward.

KẾT QUẢ

```
FP16 of 1e-8: 0.0
FP16 of 1e-8 * 65536: 0.00065517
unscaled in FP32: 9.9972e-09
```

Gradient nhỏ không bị làm tròn về 0.

Khoảng số của FP16



Gradient 10^{-8} nằm dưới số dương nhỏ nhất của FP16 nên thành 0; GradScaler nhân 65536 đưa nó vào khoảng biểu diễn được ($6.55 \cdot 10^{-4}$), rồi chia lại ở FP32.

HỎI NHANH

GradScaler nhân loss với 65536 trước khi backward. Vì sao bước cập nhật không lớn gấp 65536 lần?

A Gradient được chia lại trước khi cập nhật

B Learning rate cũng tự chia cho 65536

C FP16 tự làm tròn gradient về giá trị thật

D Hệ số chỉ nhân vào con số loss in ra log

Thứ tự một step AMP

Autocast forward và loss.

`scaler.scale(loss).backward()` → `unscale_`.

Clip nếu cần → `scaler.step` → `update`.

Clip sau `unscale_` để ngưỡng có đúng nghĩa.

Recipe đo trên GPU

Tệp gpu_probe.py, chạy riêng trên máy có CUDA.

Lệnh mặc định: kiểm môi trường, `CUDA_READY` hoặc `CPU_ONLY`.

`--run --precision fp32` hoặc `amp`: benchmark trên GPU.

Không suy ra tốc độ từ tên card: đo trên đúng máy được cấp.

Đo công việc đã hoàn tất

Warmup trước đoạn đo.

Synchronize trước và sau đồng hồ CPU.

Reset peak memory sau warmup.

JSON: ms/step, samples/s, peak allocated/reserved.

Chỉ gồm train loop trên batch đã ở GPU.

I/O và CPU→GPU cần đo riêng.

HỎI NHANH

Đo thời gian chạy CUDA bằng `time.perf_counter()` của CPU mà không synchronize. Con số đo được có thể phản ánh điều gì?

A Thời gian GPU thực sự chạy hết các kernel

B Thời gian chép dữ liệu từ ổ đĩa lên GPU

C Thời gian đã tự trừ đi các bước khởi động

D Phần lớn chỉ là thời gian CPU gửi lệnh đi

Ước lượng ngân sách

, : 391 step/epoch.

Nếu s: 20 epoch s phút.

Còn validation, I/O, checkpoint, khởi động.

Cộng validation vào



20 epoch × 391 step × 0.12 s, cộng 20 lần validation 15 s

Ước lượng ngân sách: train 20 × 391 × 0.12 s ≈ 15.6 phút, cộng validation 5 phút đã vượt 20 phút trước khi tính đọc dữ liệu và checkpoint.

HỎI NHANH

Tiếp ví dụ trên, với đã đo trên chính máy được cấp: phần train mất khoảng 15.6 phút, ngân sách là 20 phút, và mỗi epoch còn có validation đo được 15 giây. Kế hoạch 20 epoch có vừa không?

A Vừa, còn dư hơn 4 phút cho phần khác

B Vừa, vì validation dùng `no_grad` nên gần như không tốn

C Không vừa, cộng validation vào đã khoảng 20.6 phút

D Chưa biết được, phải đo lại trên T4

Khi đi thi

- Overfit một batch, assert shape và `isfinite` trước khi chạy dài.
- Ô đầu tiên: `torch.cuda.is_available()`, tên GPU, phiên bản PyTorch.
- Ngân sách: + validation, I/O, dư một lần chạy lại.
- OOM: giảm batch/độ dài, AMP, accumulation chia loss .
- Lưu `state_dict` mỗi epoch, nộp sớm từ checkpoint tốt nhất.

04

Luyện đề

LUYỆN ĐỀ 1

Accumulation 4 microbatch, có dòng A là `loss = loss / accumulation_steps` ngay trước `backward()`. Vì sao cần dòng A?

A Để gradient cộng dồn bằng đúng gradient của cả batch lớn

B Để tránh tràn số khi gọi backward nhiều lần liên tiếp

C Không bắt buộc, dòng A chỉ đổi con số loss in ra log

D Để bù cho thống kê BatchNorm tính trên microbatch nhỏ

ĐỀ THI NOAI 2026 Singapore · Chung kết · Câu 11

LUYỆN ĐỀ 2

Mixed precision: cách B là `scale → backward → step → update`, chạy tốt. Khi nào phải thêm `unscale_` trước `step`, tức cách C?

A Khi mạng rất sâu, để chống gradient bùng nổ

B Khi có clip gradient, để ngưỡng clip đúng nghĩa

C Khi batch rất nhỏ làm gradient dao động mạnh

D Chỉ khi debug, để in ra độ lớn gradient thật

ĐỀ THI NOAI 2026 Singapore · Chung kết · Câu 16

LUYỆN ĐỀ 3

Có 140 giờ GPU, cần 90 epoch. Batch 256: 76.5%, 8 giờ mỗi epoch. Batch 2048: 75.8%, 1.5 giờ. Batch 8192: 73.2%, 0.5 giờ. Chọn cách nào?

A Batch 256, chạy tới khi hết ngân sách

B Batch 8192 đủ 90 epoch, chỉ hết 45 giờ

C Accumulation 4 × 2048, tốc độ của 8192 với chất lượng của 2048

D Batch 2048 với LARS và warmup, chạy đủ 90 epoch

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 295

Chốt ba điều

Một batch phải học được trước khi đo tốc độ.

VRAM = tham số + gradient + optimizer + activation; batch chỉ chạm activation.

Accumulation chia loss ; AMP unscale trước clip; số đo lấy từ đúng máy.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; đổi số microbatch, kiểm gradient vẫn trùng batch lớn.

Có GPU: chạy `gpu_probe.py` cả fp32 và amp, so ms/step và peak VRAM.